

Which Machine Learning Algorithm Should I Use?

A business-case guide for beginners — from business problem → to the right algorithm → to the result you can expect.

BEGINNER FRIENDLY

PRACTICAL & APPLIED

What You Will Be Able to Do After This Session

By the end of this session, you will have a practical, working toolkit for approaching real-world machine learning problems with confidence. These are skills you can immediately apply on the job or in your own projects.

1

Identify the Problem Type

Read any business problem and recognise what type of ML challenge it represents — regression, classification, clustering, and more.

2

Pick and Justify an Algorithm

Select a suitable algorithm and explain clearly why it fits the problem, drawing on data availability, output type, and interpretability needs.

3

Understand the Mechanics

Describe what your chosen algorithm does, step by step, in plain language — no jargon required.

4

Interpret Outputs and Evaluate Quality

Know what output to expect from each algorithm and which metrics to use to judge whether the model is performing well.

5

Connect to the Real World

Name real companies and industries that use each algorithm today — grounding every concept in practical, commercial context.

Warm-Up Poll

Which of these do you think use machine learning? Raise your hand for each.

1. Gmail → Spam Folder

An email is silently moved out of your inbox because the system judged it unwanted.

2. Netflix Recommendations

The platform suggests a show you have never searched for, based on what you have watched.

3. Bank Loan in Minutes

A personal loan application is approved or rejected automatically, almost instantly.

4. Google Maps ETA

The app estimates your arrival time based on live traffic, history, and route data.

5. A Calculator

You type 4 + 7 and it shows 11. No data, no learning, no model — just fixed rules.

Answer: Items 1 to 4 all use ML. The calculator follows fixed rules written by a programmer — it does not learn from data and cannot improve with experience. This distinction is the heart of what makes ML different.

What Is Machine Learning?

The Core Idea

A conventional program follows rules a programmer writes by hand. A machine learning program learns the rules itself by studying past examples.

You give it data. It finds the patterns. It uses those patterns to make predictions on data it has never seen before.

Key Vocabulary

Features

The input columns — the information available about each record. In a house price model: area, number of bedrooms, location, building age.

Target

The correct answer the model is trying to predict. In a house price model: the actual sale price. In a loan model: repaid or defaulted.

Model

The mathematical relationship the algorithm works out between the features and the target. Once trained, it can predict the target for new records.

Class discussion: In a loan application, what would the features be? What would the target be? Think about what information a bank already holds about an applicant.

The Three Types of Machine Learning

Almost every ML problem falls into one of three broad families. Understanding which family your problem belongs to is the first step to choosing the right algorithm.

Supervised Learning

Data: Every row has a correct answer (a label).

Goal: Learn the relationship between features and the label, then predict the label for new rows.

Examples: Predicting house prices, approving loans, detecting fraud, classifying spam.

Unsupervised Learning

Data: No correct answer is given — rows are unlabelled.

Goal: Find hidden structure, groups, or patterns in the data on its own.

Examples: Customer segmentation, grouping stores, finding products bought together, reducing survey dimensions.

Reinforcement Learning

Data: No fixed dataset; the model tries actions and receives rewards or penalties.

Goal: Learn which sequence of actions produces the best long-term result.

Examples: Game-playing AI (AlphaGo), robotics, self-driving car steering, ad bidding systems.

This session focuses on **supervised** and **unsupervised** learning — the two families you will encounter most often in business settings.

Five Questions to Ask for Any Business Case

Before reaching for an algorithm, ask these five diagnostic questions. They will guide you to the right problem type every time — and save you from building the wrong model entirely.

1

What does the business want to know or decide?

Define the outcome clearly. "Improve customer satisfaction" is not specific enough. "Predict which customers will cancel in the next 30 days" is.

2

Do we have past data with the correct answer?

Yes → supervised learning. No correct labels exist → unsupervised learning. This is your first and most important fork in the road.

3

Is the answer a number or a category?

Number (price, quantity, temperature) → regression. Category (yes/no, which group, which plan) → classification.

4

Does the business need to know why?

Regulators, doctors, and customers often require an explanation. If yes, prefer simpler, interpretable models over black-box ones — even if the black box is slightly more accurate.

5

How much data do we have, and how clean is it?

Some algorithms (neural networks) need large, clean datasets. Others (Naive Bayes, KNN) work with small data. Data quality is often the deciding factor in practice.

Problem Type Map

Match the business question to the right problem type, then choose an algorithm from the corresponding family. This map covers the most common scenarios you will encounter in practice.

The Business Asks...	Problem Type	Common Algorithms
"How much?" or "How many?"	Regression	Linear Regression, Random Forest, Gradient Boosting
"Yes or no?" or "Which category?"	Classification	Logistic Regression, Decision Tree, Random Forest, SVM, Naive Bayes, KNN
"What natural groups exist?"	Clustering	K-Means, Hierarchical Clustering
"Is this unusual?"	Anomaly Detection	Isolation Forest
"What is bought together?"	Association Rules	Apriori
"What will happen next month?"	Time Series Forecasting	ARIMA, Prophet
"What should we suggest to this user?"	Recommendation	Collaborative Filtering
"What is in this image?"	Image Recognition	Neural Networks (CNN)

Quick Quiz: Regression or Classification?

Regression — the answer is a continuous number along a scale. **Classification** — the answer is one of a fixed set of named categories. This distinction determines which family of algorithms you will use.

Decide for Each Case

- → Predict tomorrow's temperature
- → Predict whether it will rain tomorrow
- → Predict a student's exam score
- → Predict whether a student will pass or fail
- → Predict which of three plans a customer will buy

Answers

1 → Regression

Temperature is a number on a continuous scale.

2 → Classification

Rain / No Rain — two discrete categories.

3 → Regression

A score of 67 or 82 is a number.

4 → Classification

Pass / Fail — two discrete outcomes.

5 → Classification

Three named plan options — a multi-class problem.

BUSINESS CASE 1

Estimating House Prices



The Situation

A property website wants to display an estimated price for every listed house — automatically, at scale, without a human valuer reviewing each listing.

Thinking It Through

Past data with correct answers?

Yes — historical sales records include the actual price achieved for each property.

Number or category?

A number — the predicted price in rupees or dollars.

Problem type

Supervised regression. Features include: area (sq ft), number of bedrooms, location/pin code, building age, floor number, amenities.

Algorithm to Start With: Linear Regression

Class discussion: Which single feature do you think affects price the most in your city — area, location, or something else?

Algorithm: Linear Regression

Linear regression is the simplest and most interpretable algorithm for predicting a number. It is almost always the right place to start for any regression problem.

How It Works

The model predicts the target by multiplying each feature by a learned weight and adding all the results together:

Price = (w_1 × area) + (w_2 × bedrooms) + (w_3 × location score) + … + base value

During training, the algorithm adjusts the weights repeatedly until the gap between predicted and actual prices is as small as possible. This gap-minimisation process is called **least squares optimisation**.

What to Expect as Output

A predicted number for each new property. Alongside the prediction, the weights themselves reveal which features drive prices up or down — and by how much. This interpretability is a major business advantage.

Strengths and Limits

✓ Strength: Explainability

Stakeholders can see exactly which features matter and by how much. No black-box decisions.

✓ Strength: Fast to Train

Trains in seconds even on large datasets. Easy to deploy and maintain.

✗ Limit: Assumes Linearity

Works poorly when the true relationship curves sharply or has complex interactions between features.

Real-World Use

Property portals such as **Zillow**, **MagicBricks**, and **99acres** use variants of linear regression to show estimated prices. Insurance companies use it to estimate claim amounts from vehicle or property data.

BUSINESS CASE 2

Will This Borrower Repay the Loan?

A bank wants to decide quickly whether to approve a personal loan — and must be able to explain every decision to regulators and customers.

Thinking It Through

Past data with correct answers?

Yes — historical loans each marked as "repaid" or "defaulted".

Number or category?

Category: default / no default. Two discrete outcomes.

Explanation needed?

Yes. Regulators require banks to justify credit decisions. A customer who is rejected has the right to ask why. A black-box model creates legal and reputational risk.

Problem type

Supervised classification, explainable.

Algorithm to Start With: Logistic Regression

Despite the name containing "regression", logistic regression is a **classification** algorithm. It outputs a probability that a record belongs to one of two categories.

Class discussion: Why would a bank reject a model that is highly accurate but cannot explain its decisions? Think about regulation, fairness, and customer trust.

Algorithm: Logistic Regression

Logistic regression is the workhorse of binary classification in regulated industries. It has been used in bank credit scorecards for decades because it combines reasonable accuracy with full interpretability.

How It Works

Like linear regression, it multiplies each feature by a learned weight. But instead of outputting the raw total, it passes that total through the **sigmoid function** — a mathematical formula that squashes any number into a value between 0 and 1.

That value is a **probability**. You then choose a cut-off threshold — commonly 0.5 — to convert the probability into a decision: above the threshold means "likely to default", below means "likely to repay".

The threshold itself can be tuned to control the trade-off between missing defaults and wrongly rejecting good customers.

What to Expect as Output

A probability for each applicant — for example, "18% chance of default" — plus weights that show which features increase or decrease the risk. This output is directly usable in a credit scorecard.

✓ Fast and Explainable

Output is a clear probability with feature-level explanations. Regulators accept it.

✗ Misses Complex Patterns

Assumes a roughly linear relationship between features and the log-odds of the outcome. Non-linear interactions require more advanced models.

Real-World Use

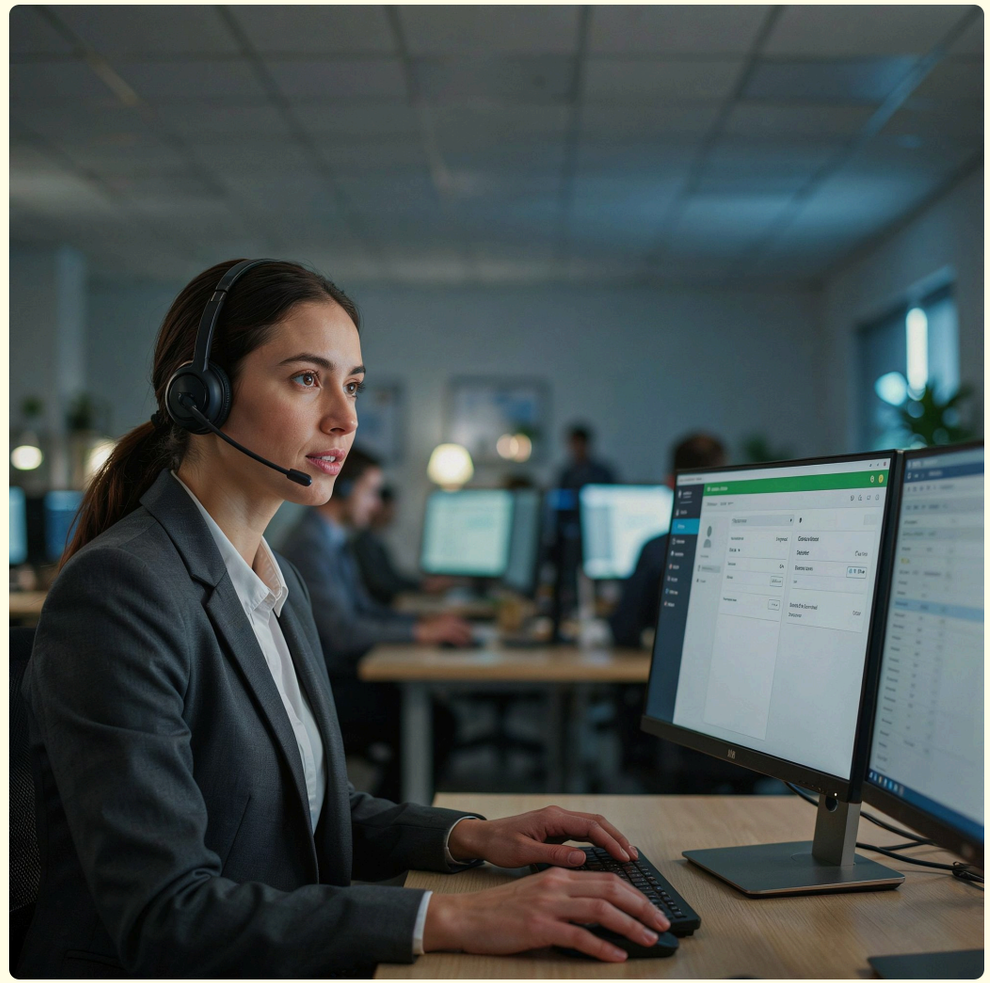
Credit scorecards at major banks globally. Also used in hospitals to estimate patient risk of readmission or complication.

BUSINESS CASE 3

Which Customers Will Leave Us?

The Situation

A telecom company loses customers every month. It wants to identify who is likely to leave — before they do — so it can offer them a better plan or targeted incentive.



Thinking It Through

Past data with correct answers?

Yes — records for customers who left (churned) and who stayed.

Number or category?

Category: leave / stay.

Explainability needed?

Yes. The marketing team wants clear, actionable rules — not just a score. They need to know *why* a customer is flagged, so they can tailor their offer.

Problem type

Supervised classification with readable rules.

Class discussion: What signs would tell you a customer is about to switch operators? Think about their contract type, usage patterns, and complaint history.

Algorithm: Decision Tree

A decision tree produces a model that looks exactly like a flowchart. Anyone in the business — not just data scientists — can read it, follow its logic, and act on it. This makes it uniquely valuable when the output must drive frontline decisions.

How It Works

The algorithm searches for the single yes/no question about the features that best separates churners from stayers. For example: "Is this a monthly contract?" might be the most powerful first question.

It then repeats the process within each resulting group, finding the next best question. It continues until each leaf of the tree is dominated by one category, or until a stopping rule kicks in.

Example Output Rule

"Monthly contract AND more than 3 complaints in the last 6 months AND no usage in the last 2 weeks → 87% likely to churn."

The marketing team can read this rule, understand it, and immediately know which customers to call and what to offer them.

Strengths and Limits

✓ Human-Readable Rules

Business stakeholders can validate the logic and spot errors without any statistical knowledge.

✓ No Feature Scaling Needed

Works directly with raw numbers and categories — no preprocessing required.

✗ Prone to Overfitting

A single deep tree memorises the training data, including its noise, and performs poorly on new customers. Pruning or ensemble methods are needed.

Real-World Use

Telecom churn analysis and retention campaigns; loan pre-screening rule sets; medical triage guidelines in hospitals and clinics.

BUSINESS CASE 4

Is This Card Transaction Fraud?

A payments company must decide in under a second whether each transaction is genuine. Fraud is rare — often fewer than 1 in 1,000 transactions — but the cost of missing it is enormous.

Thinking It Through

Past data with correct answers?

Yes — transactions labelled fraud or genuine by the investigations team.

Number or category?

Category: fraud / genuine.

Special challenge

The data is highly imbalanced. If a naive model predicts "genuine" for every single transaction, it achieves 99.9% accuracy — yet misses every fraud case entirely. Accuracy alone is a useless metric here.

Problem type

Supervised classification, high accuracy needed on the rare class.

Algorithm to Start With: Random Forest

A random forest handles imbalanced data better than a single tree and produces dramatically more reliable predictions on unseen transactions.

⚠ **Class discussion:** If a model predicts "genuine" for every transaction, it is 99.9% accurate. Why is this model worthless for the business? What metric should we use instead?

Algorithm: Random Forest

Random forest is one of the most reliable and widely deployed algorithms in industry. It addresses the core weakness of a single decision tree — overfitting — by combining the votes of many trees, each trained differently.

How It Works

01

Bootstrap Sampling

Each tree is trained on a different random sample of the training data — some rows are included multiple times, others are left out entirely.

02

Random Feature Selection

At each split in a tree, only a random subset of features is considered. This forces diversity — different trees learn different aspects of the data.

03

Majority Vote

For a new transaction, all trees give a vote. The category that receives the most votes becomes the final prediction. Errors made by individual trees cancel out across the ensemble.

What to Expect as Output

A category (fraud / genuine) for each transaction, plus a probability score. The model also produces a **feature importance ranking** — telling you which transaction attributes (e.g. unusual location, transaction size, time of day) most strongly signal fraud.

✓ High Accuracy, Little Tuning

Works well out of the box on most structured datasets. Robust to noisy features.

✗ Less Interpretable

Harder to explain than a single tree. Not the first choice when regulation demands full transparency.

Real-World Use

Fraud screening at Visa, Mastercard, and PayPal; insurance claim checks; credit risk models at retail banks.

BUSINESS CASE 5

How Many Units Will Each Store Sell?



The Situation

A retail chain needs to forecast sales for thousands of products across hundreds of stores to plan stock levels, logistics, and staffing. Under-forecasting leads to empty shelves and lost sales; over-forecasting leads to waste and storage costs.

Thinking It Through

Past data with correct answers?

Yes — years of detailed sales records at product-store-day level.

Number or category?

A number — units sold per period.

Complexity

Many features interact in non-linear ways: price, promotional discounts, festival season, weather, store location, and competitor activity all affect demand simultaneously.

Problem type

Supervised regression with complex, interacting features.

Class discussion: What happens to a store if it forecasts too low? Too high? Which error is worse in a supermarket versus a fashion retailer?

Algorithm: Gradient Boosting

Gradient boosting — implemented in popular libraries as **XGBoost** and **LightGBM** — is the top-performing algorithm on structured (tabular) data in most real-world benchmarks and competitions. It sacrifices some interpretability for substantially higher accuracy.

How It Works — Sequential Correction

Tree 1: First Attempt

The first small decision tree makes a rough prediction for every record. It will make many errors — this is expected.

Tree 2: Fix the Errors

The second tree is trained exclusively on the errors left by Tree 1 — learning where the first attempt went wrong.

Trees 3…N: Iterate

Each new tree targets the residual errors of all previous trees combined. Hundreds of trees are built this way.

Final Prediction

The final prediction is the sum of all trees' contributions — a highly refined, highly accurate result.

Strengths and Limits

✓ Best Accuracy on Tabular Data

Consistently wins Kaggle competitions and industry benchmarks on structured row-column datasets.

✓ Handles Mixed Features

Works seamlessly with numbers, categories, missing values, and non-linear interactions.

✗ Needs Careful Tuning

Hyperparameters (learning rate, tree depth, number of trees) must be tuned. Without care, it memorises training data.

Real-World Use

The **M5 Forecasting Competition** — based on Walmart's actual sales data — was won by gradient boosting models. Widely used for pricing optimisation, insurance risk scoring, and click-through-rate prediction in advertising.

BUSINESS CASE 6

Spam or Not Spam?

An email provider wants to automatically move unwanted emails out of users' inboxes. Millions of emails arrive every hour — the model must be extremely fast and work well even on limited historical data.

Thinking It Through

Past data with correct answers?

Yes — emails that users have previously marked as spam, plus emails confirmed as genuine.

Number or category?

Category: spam / not spam.

Special challenge

The input is raw text — which becomes thousands of features (one per unique word). Standard numeric algorithms do not naturally handle this. Speed is critical: decisions must be made in milliseconds.

Problem type

Supervised text classification, speed-critical.

Class discussion: Name three words or phrases that immediately make you suspect an email is spam. Now think — how could a spammer try to dodge a word-based filter?

Algorithm to Start With: Naive Bayes

Naive Bayes is one of the oldest ML algorithms still in active use — and for good reason. On text data with limited training examples, it is remarkably effective and almost instantaneous to run.

Algorithm: Naive Bayes

Naive Bayes applies Bayes' theorem to estimate the probability that an email belongs to each category, based on the words it contains. It earns the label "naive" because it assumes each word's presence is independent of every other word — a simplification that is technically wrong but practically very effective.

How It Works

01

Learn Word Frequencies

From thousands of past emails, the model counts how often each word appears in spam versus genuine mail. "Free" appears in 40% of spam but only 2% of genuine mail. "Meeting" is the reverse.

02

Combine Word Probabilities

For a new email, it multiplies together the individual probabilities of every word it contains — for spam and for genuine — giving an overall probability for each category.

03

Choose the Winning Category

The category with the higher overall probability wins. The output is a label (spam / not spam) plus a confidence score.

✓ Works with Little Data

Even a few hundred labelled examples are enough to produce a useful spam filter.

✓ Extremely Fast

Classifies millions of emails per second. Trivial to scale.

✗ Ignores Word Order

"Not good" and "good, not bad" contain similar words but opposite meanings. Naive Bayes cannot tell them apart.

Real-World Use

Early spam filters, including **SpamAssassin**, pioneered this approach. Naive Bayes is still used today for routing customer support emails, classifying news articles by topic, and labelling social media posts.

BUSINESS CASE 7

Is This Patient at Risk of Diabetes?

The Situation

A diagnostic lab wants to automatically flag patients who may be diabetic based on routine blood test readings — glucose levels, BMI, insulin, blood pressure — so that a doctor can prioritise reviewing them.

Thinking It Through

Past data with correct answers?

Yes — past patients with confirmed diagnosis labels: diabetic or not diabetic.

Number or category?

Category: at risk / not at risk.

Dataset size

Small — a few thousand patients. Complex algorithms requiring massive data are overkill and will likely overfit.

Core intuition

Patients with very similar test readings tend to share the same diagnosis. Finding similar past patients is a natural fit.

Problem type

Supervised classification on a small dataset.



Class discussion: Which test readings would you compare between two patients to decide if they are "similar"? Does the scale of the measurement matter?

Algorithm: K-Nearest Neighbours (KNN)

KNN is one of the most intuitive algorithms in all of machine learning. It makes no assumptions about the shape of the data and requires no training phase — it simply memorises the past and reasons by analogy.

How It Works

01

Choose K

Select a number K — for example, 5. This determines how many "neighbours" will vote on each new patient's classification.

02

Find the Closest Past Patients

For a new patient, measure the numeric distance between their readings and every past patient in the dataset. Select the 5 with the smallest distance.

03

Majority Vote

If 4 of those 5 neighbours were diabetic, the model predicts "diabetic" for the new patient. The category is determined by the majority label among the K neighbours.

Important: Because "distance" is used, all features must be on the same scale before running KNN. A feature measured in thousands (e.g. income) will dominate over one measured in units (e.g. number of children) unless you normalise first.

Strengths and Limits

✓ No Training Step

The model simply stores all past records. Adding new labelled patients updates the model instantly — no retraining needed.

✓ Easy to Explain

"Your prediction is based on 5 past patients whose readings were closest to yours." Any doctor can understand this.

✗ Slow on Large Data

For every new patient, it must compare them to every past patient. With millions of records, this becomes computationally expensive.

✗ Requires Feature Scaling

Raw features at very different scales will produce misleading distances. Preprocessing is essential.

Real-World Use

"Similar items" product suggestions, basic medical screening studies, and handwriting recognition baseline systems.

BUSINESS CASE 8

Sending Support Tickets to the Right Team

A software company receives thousands of support tickets every day. Manually reading and routing each one to the Billing, Technical, or Account Management team is slow and error-prone. The company wants this done automatically.

Thinking It Through

Past data with correct answers?

Yes — archived tickets, each stamped with the team that handled it.

Number or category?

Category with three options: Billing / Technical / Account.

Special challenge

Input is unstructured text. When converted to a bag-of-words representation, each ticket becomes a vector with thousands of dimensions — one per unique word in the vocabulary.

Problem type

Supervised text classification with very high-dimensional input.

Class discussion: A ticket reads: "I was charged twice and now I cannot log in to my account." Which team should receive it — Billing, Technical, or both? How would you handle tickets that belong to more than one category?

Algorithm to Start With: Support Vector Machine (SVM)

SVM is particularly well-suited to high-dimensional text data because it finds the widest possible decision boundary between categories — even in thousands of dimensions.

Algorithm: Support Vector Machine (SVM)

SVM approaches classification as a geometric problem: find the dividing line (or hyperplane) that separates two categories with the maximum possible gap between them. A wider gap means greater confidence and better generalisation to new data.

How It Works

Each support ticket is represented as a point in a high-dimensional space — one dimension per word. SVM looks for the dividing hyperplane that:

- Correctly separates the two categories
- Maximises the distance (margin) from the boundary to the nearest points of each category

Those nearest points are the **support vectors** — the only records that actually determine where the boundary sits. All other records are irrelevant to the boundary's position.

When a straight boundary cannot work — categories are interleaved — SVM applies a **kernel trick** to map the data into a higher dimension where a straight boundary becomes possible. No extra data is needed; this is a mathematical transformation.

Strengths and Limits

✓ Excellent on Text Data

Performs very well in high-dimensional spaces where most algorithms struggle. Often the best off-the-shelf text classifier.

✓ Robust to Overfitting

The maximum-margin principle naturally resists memorising the training data, especially in high dimensions.

✗ Slow on Very Large Data

Training time grows quickly with millions of rows. Deep learning methods are now preferred at very large scale.

✗ Hard to Explain

The decision boundary is mathematical and difficult to express in plain language to a non-technical audience.

Real-World Use

Text categorisation, handwritten digit recognition for postal codes (widely deployed in the 1990s), classifying biological samples and genomic sequences.

BUSINESS CASE 9

What Types of Customers Do We Have?



The Situation

An online retailer wants to send different promotional offers to different kinds of customers — but has no predefined customer categories. No one has labelled customers as "loyal", "bargain hunter", or "at risk". The business wants to discover the natural groupings that exist in the data.

Thinking It Through

Past data with correct answers?

No — nobody has labelled the customers. This shifts us to unsupervised learning.

Goal

Find groups of customers who behave similarly, based on the RFM framework: Recency (how recently they bought), Frequency (how often), Monetary value (how much they spend).

Problem type

Clustering (unsupervised).

Class discussion: What distinct customer groups would you expect to find in a shopping app — before running any algorithm? What names would you give them?

Algorithm: K-Means Clustering

K-Means is the most widely used clustering algorithm. It is fast, scalable, and produces clear, actionable segments that non-technical teams can immediately name and work with.

How It Works

01

Choose K

Decide on the number of groups — for example, 4 customer segments. Choosing K well is a skill in itself; techniques like the "elbow method" help.

02

Place Centre Points

Randomly position 4 "centroid" points in the feature space.

03

Assign Every Customer

Each customer is assigned to the nearest centroid, based on the distance between their feature values and the centroid's position.

04

Update Centroids

Each centroid moves to the average position of all the customers currently assigned to it.

05

Repeat Until Stable

Steps 3 and 4 repeat until assignments stop changing — typically within 10 to 50 iterations.

What to Expect as Output

A segment number (1 to 4) for every customer. The data team then analyses each segment and gives it a business-meaningful name:

Segment 1

Frequent high spenders → "VIP Loyalists"

Segment 2

Bought once, long ago → "Dormant Customers"

Segment 3

Buys often but low value → "Bargain Hunters"

Segment 4

Recent first-timers → "New Prospects"

✓ Fast and Scalable

Works on millions of customers. Easy to re-run as new purchase data arrives.

✗ Must Choose K

The wrong K produces misleading segments. Also assumes roughly spherical, equal-sized groups.

Real-World Use

Customer segmentation across retail, telecom, and banking; grouping delivery locations into efficient zones for logistics routing.

BUSINESS CASE 10

Grouping Stores When We Don't Know How Many Groups

A supermarket chain wants to group its 200 stores by sales pattern — but management has no prior view on how many meaningful groups should exist. They want to explore the data at multiple levels of detail before committing to a final segmentation.

Thinking It Through

Past data with correct answers?

No — unsupervised. No store has been pre-labelled with a group.

Special requirement

Management wants to see how stores relate at *different levels* — both broad macro-groups and detailed sub-groups — before deciding where to cut.

Problem type

Clustering with a hierarchy — K is unknown.

Class discussion: Why might two stores in completely different cities sell in a similar pattern? Think about demographics, store size, and neighbourhood type.

Algorithm to Start With: Hierarchical Clustering

Unlike K-Means, hierarchical clustering does not require you to specify K in advance. It builds a full tree of relationships that you can "cut" at any level.

Algorithm: Hierarchical Clustering

Hierarchical clustering builds a complete map of how every store relates to every other store, from the most similar pair all the way up to a single global group. The result is a tree diagram called a **dendrogram** — one of the most visually informative outputs in all of unsupervised learning.

How It Works (Agglomerative / Bottom-Up)

Start: Every Store Is Its Own Group

With 200 stores, you begin with 200 singleton clusters.

Merge the Closest Pair

Find the two most similar stores (or groups) and merge them into one. Now you have 199 clusters.

Repeat

Continue merging the closest remaining pair at each step. The process runs until all stores are in one cluster.

Cut the Dendrogram

The full hierarchy is drawn as a tree. You cut it at the level that produces a useful and interpretable number of groups — 3 broad regions, or 8 store archetypes, as the business needs.

Strengths and Limits

✓ No Need to Pre-specify K

The full hierarchy is computed once. You explore the dendrogram and choose the cut that makes business sense — without rerunning the algorithm.

✓ Shows Relationships at All Levels

You can see which stores are most similar to each other, not just which group they belong to — valuable for explaining results to management.

✗ Slow on Large Datasets

Comparing every pair of items becomes computationally expensive as the number of records grows into the tens of thousands.

Real-World Use

Grouping genes with similar expression patterns in biology research; grouping news articles or legal documents by topic; store and product grouping for retail planning.

BUSINESS CASE 11

Spotting a Machine Before It Fails



The Situation

A factory has sensors on every machine recording temperature, vibration, and pressure every second. It wants an automated alert when sensor readings look unusual — before the machine breaks down, causing costly downtime.

Thinking It Through

Past data with correct answers?

Mostly no. Machine failures are rare events and have rarely been systematically labelled in historical records.

Goal

Find readings that look different from what the machine normally produces — without knowing in advance what "different" looks like.

Problem type

Anomaly detection (unsupervised).

Class discussion: Why can't we simply set a fixed rule like "alert if temperature is above 90°C"? Think about what varies between machines, seasons, and operating conditions.

Algorithm: Isolation Forest

Isolation Forest takes a counterintuitive approach to anomaly detection: rather than defining what "normal" looks like and flagging everything else, it directly measures how easy it is to isolate each record from the rest of the data. Anomalies isolate easily; normal records do not.

How It Works

01

Random Splitting

The algorithm randomly picks a feature and a random split value, dividing the data into two halves. It repeats this many times, building a forest of random trees.

02

Count the Splits Needed

For each record, it counts how many splits are needed before the record is isolated in its own partition. An anomalous reading gets isolated after very few splits — because few other records have similar values.

03

Compute the Anomaly Score

Records that consistently require fewer splits receive a higher anomaly score. Normal records need many splits to isolate and receive a low score.

04

Set the Alert Threshold

You choose the score above which to trigger an alert. This threshold can be calibrated based on the acceptable false alarm rate.

Strengths and Limits

✓ No Labels Needed

Works entirely unsupervised — ideal when failure events are rare and have never been systematically recorded.

✓ Fast on Large Data

Scales well to millions of sensor readings per day. Suitable for real-time streaming applications.

✗ No Explanation

The score tells you something is unusual but not which specific sensor reading or combination of readings caused the alert. Additional analysis is needed.

Real-World Use

Predictive maintenance in manufacturing (automotive, aerospace, energy); unusual financial transaction detection; network intrusion and cybersecurity monitoring.

BUSINESS CASE 12

Which Products Are Bought Together?

A supermarket wants to discover which products customers naturally buy on the same shopping trip — to optimise shelf placement, cross-promotions, and bundled offers. No prediction is needed; the goal is to uncover hidden purchasing patterns.

Thinking It Through

Past data with correct answers?

No target to predict — this is pattern discovery in millions of transaction receipts.

Goal

Find combinations of items that appear on the same receipt more often than chance would predict.

Problem type

Association rule mining (unsupervised).

Three Key Measures

Support

How often does this combination appear across all transactions? A rule with low support affects very few customers.

Confidence

When a customer buys A, what fraction also buy B? High confidence means B is a reliable recommendation for A-buyers.

Lift

How much more often do A and B appear together than pure chance predicts? Lift > 1 means a genuine relationship; lift = 1 means coincidence.

Class discussion: Think of your last grocery shopping trip. Which two items did you buy that you almost always buy together? Would placing them in opposite corners of the store increase or decrease your total spend?

Algorithm: Apriori

Apriori uses a clever pruning strategy — "if a pair is rare, any larger set containing it must also be rare" — to make the otherwise enormous search space manageable, even with thousands of distinct products.

How It Works

01

Count Single Items

Count how often every individual product appears across all receipts. Drop products that appear fewer times than the minimum support threshold.

02

Form Pairs

From the surviving items, form all possible pairs and count their frequency. Drop rare pairs. This uses the Apriori principle: if an item is rare alone, any pair containing it is also rare.

03

Extend to Triplets and Beyond

Continue to groups of three, four, and more — always pruning rare combinations immediately to keep the computation tractable.

04

Generate Rules

Convert frequent itemsets into "if A then B" rules. Filter by minimum confidence and minimum lift to keep only genuinely useful rules.

Example Output Rules

Bread + Butter → Jam

Confidence: 62% | Lift: 3.1 — these three items appear together 3× more often than chance. Strong candidate for a "breakfast bundle" promotion.

Nappies → Beer

The famous (and real) supermarket finding from the 1990s: a strong association, believed to reflect parents shopping for the week on a Friday evening.

Real-World Use

Shelf layout and cross-promotion design in supermarkets and convenience stores; "Frequently Bought Together" recommendations on Amazon and Flipkart; pharmacy cross-sell suggestions based on prescription patterns.

BUSINESS CASE 13

Too Many Columns to Work With

The Situation

A company runs a customer satisfaction survey with 120 questions. Many questions overlap — "I find the product easy to use" and "The product does what I expect" may receive nearly identical answers from every respondent. Models built on all 120 columns are slow to train, prone to instability, and difficult to visualise.



Thinking It Through

Is this a prediction task?

Not directly — this is a **data preparation task**. The goal is to simplify the dataset before feeding it into another algorithm.

Goal

Reduce 120 columns to a much smaller number while retaining as much meaningful information as possible.

Problem type

Dimensionality reduction (unsupervised).

Class discussion: If two survey questions always receive nearly identical answers from every respondent, what does that tell us about the information they each contribute? Do we need both?

Algorithm: Principal Component Analysis (PCA)

PCA is the most widely used technique for dimensionality reduction. It creates a smaller set of new columns — called **principal components** — that are linear combinations of the original columns and capture the maximum possible variation in the data.

How It Works

01

Find Correlated Columns

PCA identifies groups of original columns that move together — columns that are essentially measuring the same underlying thing.

02

Create Principal Components

Each component is a weighted combination of the original columns. The first component captures the largest possible share of total variation; each subsequent component captures the next largest share, subject to being uncorrelated with all previous components.

03

Select How Many to Keep

A scree plot shows how much variation each component explains. You keep enough components to account for — for example — 90% of the total variation. Typically this means going from 120 columns to 10 or 15.

What to Expect as Output

A new, smaller dataset with far fewer columns that retains most of the original information. For example: 120 survey columns reduced to 10 components that preserve 90% of the variation. Models trained on this reduced dataset are faster, more stable, and easier to visualise in 2D or 3D scatter plots.

✓ Removes Redundancy

Eliminates the noise introduced by correlated, overlapping features. Reduces the risk of overfitting.

✗ Hard to Interpret New Columns

Each principal component is a blend of many original questions. You can no longer say "this axis represents customer satisfaction with delivery speed."

Real-World Use

The pioneering "eigenfaces" technique in face recognition research; compressing sensor data in IoT applications; preparing survey and genomic data before running clustering algorithms.

BUSINESS CASE 14

How Much Electricity Will the City Need Next Month?

A power company must plan fuel procurement, staffing, and grid capacity weeks in advance. It holds five years of hourly electricity demand data and needs reliable forecasts broken down by time of day, day of week, and season.

Thinking It Through

Past data with correct answers?

Yes — five years of actual hourly demand readings.

Number or category?

A number — megawatts demanded per hour.

Critical feature

The **order of the data matters fundamentally**. Demand at 8am on a Monday is not independent of demand at 7am on the same Monday. Standard regression algorithms ignore this time dependency entirely.

Problem type

Time series forecasting.

Class discussion: At what time of day is electricity demand highest in your city? In which month? Why do you think those patterns exist? How would a major festival or a cricket match affect demand?

Algorithm to Start With: ARIMA or Prophet

Both methods are designed specifically for data where observations are ordered by time and where past values predict future ones.

Algorithms: ARIMA and Prophet

Both ARIMA and Prophet decompose a time series into meaningful components and forecast by combining those components. They differ in their approach and in how they handle external factors like holidays.

The Decomposition Approach

Trend

The long-term direction of the data — is demand growing, shrinking, or flat over years?

Seasonality

Patterns that repeat at fixed intervals — daily peaks in morning and evening, weekly troughs at weekends, annual summer and winter surges.

Remainder

What is left after removing trend and seasonality — random noise and one-off events.

ARIMA

Predicts the next value from a weighted combination of recent past values and recent prediction errors. Requires the analyst to choose parameters (p, d, q) that control how far back to look. Best for data with a single, well-defined seasonal pattern.

Prophet

Developed at **Meta (Facebook)** for their own internal forecasting and released publicly in 2017. Prophet fits trend, seasonality (multiple levels simultaneously — daily, weekly, yearly), and user-supplied holiday effects as separate components and adds them together. It is far easier to configure than ARIMA and handles missing data and outliers gracefully.

✓ Needs Only Dates and Values

No feature engineering required. You supply timestamps and measurements; the algorithm handles the rest.

✓ Outputs Uncertainty Ranges

Each forecast comes with upper and lower bounds — valuable for planning decisions that involve risk.

✗ Cannot Foresee Surprises

A lockdown, a sudden heatwave, or a major policy change will not be forecast correctly unless supplied as an external feature.

Real-World Use

Energy demand forecasting; call-centre and hospital staffing; retail sales and inventory planning.

BUSINESS CASE 15

What Should We Suggest to This User?



The Situation

A video streaming service has tens of thousands of titles. Showing every user the same list would be useless. The service wants to personalise the first row of suggestions for each user — increasing watch time and reducing the chance that a frustrated user cancels their subscription.

Thinking It Through

What data do we have?

A large matrix: which user watched, rated, or added which title to their watchlist. We may know very little about a user personally, but we know a great deal about their behaviour.

Goal

Predict which unseen titles each user is most likely to enjoy, then rank and display those titles.

Problem type

Recommendation (collaborative filtering).

Class discussion: Open any streaming or shopping app right now. Look at your personalised homepage. What behaviour of yours do you think the app used to generate those suggestions?

Algorithm: Collaborative Filtering

Collaborative filtering is the backbone of modern recommendation systems. The key insight is simple: people who agreed on what they liked in the past will likely agree in the future. No knowledge about the content of the items themselves is needed.

Two Approaches

User-Based Filtering

Find users whose past viewing behaviour matches yours most closely. Then recommend titles those users loved that you have not yet watched. "Users like you also watched…"

Item-Based Filtering

Find titles that tend to be watched by the same people. If many users who watched Film A also watched Film B, recommend Film B to anyone who has just finished Film A. "Because you watched…"

What to Expect as Output

A personalised ranked list of titles for each user, refreshed as their behaviour changes. The list can be computed in advance (batch) or in real time as the user browses.

The Cold Start Problem

Collaborative filtering has one significant weakness: it relies entirely on historical behaviour. A brand-new user who has not yet watched anything, or a newly added title that nobody has watched yet, cannot benefit from collaborative filtering. This is called the **"cold start" problem**. In practice, streaming services combine collaborative filtering with **content-based filtering** (using the genre, cast, and director of a title) to handle new users and new content.

✓ Discovers Non-Obvious Connections

Finds suggestions a rule-based system would never think of — linking users by taste rather than by demographics.

✗ Cold Start

Works poorly for new users and new items with no interaction history.

Real-World Use

Amazon published its item-to-item collaborative filtering method in 2003 — it is still the basis of their "Frequently Bought Together" suggestions. **Netflix** ran the Netflix Prize competition (2006–2009) to improve its rating predictions. **Spotify** uses collaborative filtering as a core component of its Discover Weekly playlist.

BUSINESS CASE 16

Finding Defective Products From Photos

A factory photographs every product coming off the assembly line and wants to automatically reject defective units — without a human inspector reviewing each photo. The system must be fast, accurate, and scalable to thousands of units per hour.

Thinking It Through

Past data with correct answers?

Yes — thousands of past photos labelled "defective" or "fine" by human inspectors.

Number or category?

Category: defective / fine.

Why standard algorithms fail

An image of 500×500 pixels has 250,000 input values. More importantly, *where* those values are positioned matters — a scratch in the top-left corner means the same thing as a scratch in the bottom-right corner. Standard tabular algorithms have no concept of spatial position.

Problem type

Image classification — requires a neural network.

Class discussion: How does a human inspector decide a product is defective? Could you write down all those rules in advance, completely and precisely? What does this tell you about why rule-based systems fail at visual inspection?

Algorithm: Neural Networks and CNNs

Convolutional Neural Networks (CNNs) are the standard algorithm for any task where the input is an image, video frame, or audio spectrogram. They learn to recognise increasingly complex visual patterns through multiple layers of learned filters.

How a CNN Works

Input Layer: Raw Pixels

The image enters as a grid of pixel values. Each pixel has three values (Red, Green, Blue). No manual feature engineering is done.

Early Layers: Edges and Colours

Small convolutional filters slide across the image, detecting edges, colour gradients, and simple textures — the basic visual vocabulary.

Middle Layers: Shapes and Patterns

Later layers combine the low-level detections into higher-level shapes: a scratch, a dent, a discolouration, a missing component.

Output Layer: Decision

The final layer combines all detected shapes into a single decision: defective (with a probability score) or fine.

What the Model Learns

Crucially, the network learns *what to look for by itself* from thousands of labelled photos — no human defines the rules. The patterns it discovers are often ones that a human could not articulate in words, which is part of why it outperforms rule-based systems.

✓ Handles Complex Visual Input

Outperforms all other algorithms on images, audio, and video. Matches or exceeds human accuracy on many visual tasks.

✗ Data Hungry

Typically needs thousands to millions of labelled examples to train well. Small datasets lead to overfitting.

✗ Hard to Explain

The internal representations are not human-readable. Increasingly, "explainability" techniques like Grad-CAM are used to highlight which parts of an image drove the decision.

Real-World Use

Photo search in Google Photos; PCB defect detection in electronics manufacturing; detecting diabetic retinopathy from retinal scans (Google research, 2016 — matching ophthalmologist accuracy).

How Do I Know If My Model Is Good?

Choosing the right evaluation metric is just as important as choosing the right algorithm. Using the wrong metric can make a useless model look excellent — as the fraud example illustrated.

For Regression (Predicting Numbers)

MAE — Mean Absolute Error

The average size of the error, measured in the same units as the target (e.g. ₹50,000 average error on house prices). Easy to explain to non-technical stakeholders. Treats all errors equally regardless of size.

RMSE — Root Mean Squared Error

Similar to MAE but penalises large errors more heavily. A model with a few catastrophic predictions will have a much higher RMSE than MAE. Use RMSE when large errors are especially costly.

R^2 — Coefficient of Determination

Ranges from 0 to 1 (higher is better). Tells you what fraction of the variation in the target the model successfully explains. An R^2 of 0.85 means the model explains 85% of the variation; 15% remains unexplained.

For Classification (Predicting Categories)

Accuracy

Share of all predictions that are correct. Misleading when categories are imbalanced (e.g. 99.9% genuine transactions). Often not the metric you want for high-stakes problems.

Precision

Of all the cases the model flagged as positive (e.g. "fraud"), what fraction were genuinely positive? High precision = few false alarms. Important when acting on a false alarm is expensive.

Recall (Sensitivity)

Of all the true positives in the data, what fraction did the model correctly catch? High recall = few missed cases. Critical when missing a real case is very costly — cancer screening, fraud, equipment failure.

Class discussion: For a cancer screening model, which matters more — precision or recall? What is the cost of a false positive (healthy patient told they might have cancer)? What is the cost of a false negative (cancer missed)?

Two Common Problems: Overfitting and Underfitting

Every data scientist encounters these two failure modes. Recognising them early — before deploying a model — is one of the most important practical skills you will develop.

Overfitting

The model has memorised the training data in too much detail — including its random noise and outliers. It performs brilliantly on data it has seen, but fails when presented with new data it has never encountered.

How to spot it: Training accuracy is very high (e.g. 99%), but test accuracy is much lower (e.g. 72%). The gap between training and test performance is the red flag.

What to do:

- Use more training data if available
- Switch to a simpler model (fewer parameters)
- Remove low-quality or irrelevant features
- Apply regularisation techniques (L1, L2, dropout)
- For trees: limit tree depth or use pruning

Underfitting

The model is too simple to capture the genuine patterns in the data. It makes the same types of errors on both training and test data because it never learned the relationship in the first place.

How to spot it: Both training accuracy and test accuracy are poor. There is no gap between the two — because the model is performing badly on everything.

What to do:

- Use a more capable algorithm (e.g. gradient boosting instead of linear regression)
- Engineer better, more informative features
- Remove constraints that are preventing the model from learning
- Train for longer (for neural networks)

Golden Rule: Always split your data into a training set and a test set *before* you begin. Judge model quality only on the test set — data the model has never seen during training. This rule is non-negotiable.

Cheat Sheet: Business Case to Algorithm

Keep this table as a quick reference. Start simple, validate on a test set, and only move to a more complex algorithm if the simpler one does not meet the business requirement.

Business Case	Problem Type	Start With	Key Reason
Estimate house price	Regression	Linear Regression	Explainable, fast
Approve or reject a loan	Classification, explainable	Logistic Regression	Probability + weights
Predict customer churn with clear rules	Classification	Decision Tree	Human-readable rules
Detect fraud	Classification, high accuracy	Random Forest	Ensemble accuracy
Forecast sales across stores	Regression, complex	Gradient Boosting	Best tabular accuracy
Filter spam emails	Text classification	Naive Bayes	Fast, low data needed
Screen patients on small data	Classification	KNN	Intuitive, no training
Route support tickets	Text classification	SVM	High-dimensional text
Segment customers	Clustering	K-Means	Fast, actionable
Group stores, K unknown	Clustering	Hierarchical Clustering	No K needed upfront
Detect abnormal sensor readings	Anomaly detection	Isolation Forest	No labels required
Find products bought together	Association rules	Apriori	"If this, then that"
Reduce too many columns	Dimensionality reduction	PCA	Remove redundancy
Forecast demand over time	Time series	ARIMA / Prophet	Dates + trend + season
Suggest items to users	Recommendation	Collaborative Filtering	Behaviour-based
Classify images for quality control	Image classification	CNN	Spatial patterns

Class Activity: You Are the Data Scientist

Work in pairs. For each of the six cases below, answer three questions: **(1) What is the problem type? (2) Which algorithm would you start with? (3) What output would you expect and how would you measure success?** Take 10 minutes, then share your answers with the group.

1

Food Delivery App

The app wants to estimate delivery time in minutes for each order, based on distance, restaurant preparation time, driver availability, and traffic conditions.

2

Insurance Fraud Flagging

An insurance company wants to flag suspicious claims — but has never systematically labelled past claims as fraudulent or legitimate. No labelled training data exists.

3

University Enrolment Prediction

A college admits students each year but wants to predict which admitted students will actually enrol, so it can manage class sizes and scholarship offers more accurately.

4

Fashion Retailer Customer Segmentation

A fashion e-commerce brand wants to divide its customer base into meaningful groups for targeted email campaigns. No predefined customer types exist.

5

Music App Song Suggestions

A music streaming app wants to suggest new songs each user is likely to enjoy, based on their listening history and the patterns of other users with similar tastes.

6

Hospital Staffing Forecast

A hospital's emergency department wants to forecast the number of patients arriving next week, to schedule nurses and doctors optimally without over- or under-staffing.

Activity: Suggested Answers

Compare your pair's answers with these suggested approaches. There is rarely one "correct" answer — the justification matters more than the choice of algorithm.

Case	Problem Type	Algorithm	Key Reasoning
Food delivery ETA	Regression	Linear Regression → Gradient Boosting	Predicting minutes (a number). Start simple; upgrade when interactions between distance, time of day, and traffic are too complex for a linear model.
Insurance fraud — no labels	Anomaly detection	Isolation Forest	No labelled fraud cases → unsupervised. Isolation Forest finds claims that look different from the normal pattern without requiring fraud labels.
Enrolment prediction	Classification	Logistic Regression	Binary outcome (enrols / does not enrol). Past admission records provide labels. Logistic regression gives a probability per student, enabling the admissions office to plan offers accordingly.
Fashion customer segments	Clustering	K-Means	No predefined groups → unsupervised. K-Means on RFM features will surface distinct customer archetypes for targeted campaigns.
Music song suggestions	Recommendation	Collaborative Filtering	Behaviour-based recommendations. No need to know song genre or lyrics — patterns across users' listening histories are sufficient.
Hospital staffing forecast	Time series forecasting	ARIMA / Prophet	Demand is ordered by time and shows weekly and seasonal patterns. Prophet handles multiple seasonality levels and holiday effects with minimal configuration.

Key Takeaways

These five principles will serve you well on every ML project — whether you are a student building your first model or a professional deploying your hundredth.

Start with the Business Question

Resist the urge to reach for an interesting algorithm first. The problem type — not the technology — should drive every decision. Understand what the business needs to know or do before opening a notebook.

Apply the Five Diagnostic Questions

Goal → labels → number or category → explainability → data size. Work through these every time. They will reliably guide you to the right family of algorithms and save you from expensive wrong turns.

Begin Simple, Escalate Deliberately

Start with the simplest adequate model — linear regression before gradient boosting, logistic regression before a neural network. A simple model that works is better than a complex model that is hard to maintain, debug, and explain. Complexity should earn its place.

Always Test on Unseen Data

Split your data before you begin. A model's performance on the test set — data it has never touched during training — is the only honest measure of how it will behave in production.

No Algorithm Is Universally Best

The No Free Lunch theorem formalises this: no single algorithm outperforms all others across all problems. The problem type, data characteristics, and business constraints decide the choice. Build your judgment by practising across many different cases.

Your next step: Pick one business case from this deck, find a public dataset for it on Kaggle or the UCI ML Repository, and build your first model. Learning by doing is irreplaceable.